

Review papers

Research on the long-term and short-term forecasts of navigable river's water-level fluctuation based on the adaptive multilayer perceptron



Tianxiang Zhou^{a,*}, Zhaobing Jiang^b, Xujian Liu^c, Kun Tan^d

^a Department of Mathematics, University of California, Los Angeles, United States

^b School of Civil Engineering, Sanjiang University, Nanjing, China

^c Department of Mathematics, Nanjing Tech University, Nanjing, China

^d Changjiang Waterway Planning, Design and Research Institute, Wuhan, China

ARTICLE INFO

This manuscript was handled by A. Bardossy, Editor-in-Chief, with the assistance of Fi-John Chang, Associate Editor

Keywords:

Navigable river
Water-level fluctuation
Deep learning
Multilayer perceptrons
Time Series

ABSTRACT

Water-level fluctuation (WLF) forecast plays an essential role in the regulations and management of the Yangtze River's navigable rivers. Traditional methods for navigable rivers' forecasts are often time-consuming and cost-inefficient, often requiring constant updates and are univariate in nature. This study utilized one of the Yangtze River's navigable rivers, Nanjing navigable river, to perform forecasts employing deep learning techniques. For the first time, Nanjing navigable river's WLF data was exploited to make multi-step univariate and multivariate time series forecasts, employing a deep learning technique, multilayer perceptron (MLP). The data consisted of 3545 days worth of WLF measurements from ten different navigable rivers and was allocated properly without data leakage into training and testing. MLP was built to take time series sequence inputs from up to ten navigable rivers on a daily basis and output fast, accurate, stable, and reliable short-term forecasts and undeviating trends for long-term forecasts. The models were also constructed in an adaptive way so they can self-update daily based on the daily measurements. The naive model was constructed as a baseline model to measure the improvement and the validity of MLP models. Univariate and multivariate adaptive models were then constructed based on the augmented data by a popular data-augmentation method, rolling windows. Hyperparameters of MLP models were optimized based on relevant testing and large scale grid-search. Both short-term and long-term forecasts showed promises of MLP model in time series forecast for navigable river water-level problems and they achieved at least 27.6% percent lower in root mean square error (RMSE) compared to the baseline. Since the total time required for the trained models to produce forecasts and self-update is at most 15 min on a local quad-core Intel Core i5 10th generation (2.0 GHz) and because of the nonnegotiable performance of the models, it was concluded that MLP is a fast, cost-efficient, and accurate alternative technique for navigable rivers WLF forecasts.

1. Introduction

Note that a list of abbreviations and symbols used are in the Appendices. Yangtze River is one of the largest rivers in the world with a drainage area of 1.8 million square-kilometers, which contributes to one fifth of China inland area (Chen and Gu, 2010). WLF of Yangtze navigable rivers are crucial to national transportation (King and Webber, 2008), ecosystem (Coops et al., 2003), irrigation (Schuurmans et al., 1999), fishery (Cohen and Radomski, 1993), flood planning (Alsdorf et al., 2000), hydropower (Li and He, 2008) etc, making navigable river WLF forecast extremely valuable. Until today, worldwide WLF forecasting focuses on large lakes for their significance in globalization, for instance, Lake Van (Altunkaynak et al., 2003). However,

forecasting WLF of artificial/non-artificial local navigable rivers may have tremendous economic benefits for local governments.

Traditional approaches of WLF forecast can be divided into theoretical methods, numerical simulations, and experimental physics. They have some major power as well as major drawbacks. Theoretical methods (Aksoy et al., 2013), based on statistics, are accurate and reliable in an ideal scenario but unable to handle multivariate WLF forecasts with high complexities in a realistic setting. Numerical simulations work by solving complex hydrodynamic equations (Kebede et al., 2006), requiring a large number of parameters (air temperature, wind level, precipitation and etc). This approach has developed rapidly in the past twenty years; yet, it is still inapplicable for areas with limited measurements. Other limitations of numerical simulations are

* Corresponding author.

E-mail address: cooltony128@gmail.com (T. Zhou).

<https://doi.org/10.1016/j.jhydrol.2020.125285>

Received 7 May 2020; Received in revised form 27 June 2020; Accepted 7 July 2020

Available online 21 July 2020

0022-1694/© 2020 Elsevier B.V. All rights reserved.

computational complexities and accuracy. Experimental physics (Fischer and Öhl, 2005) is the most costly approach and can only be applied to a locality for mechanism discoveries. These limitations led to the development of neural networks that rely on completely different assumptions and possess the capability to accurately predict WLF with limited resources. Neural networks can also take other variables (other cities' navigable rivers' WLF) into account and build multivariate models, as other variables might have strong interactions with the studied city.

In the last decades, artificial neural networks or ANNs have shown promises of time series forecast in diverse fields: surface water-level forecasting (Piasecki et al., 2017), short-term traffic forecasting (Zhao et al., 2017), energy consumption forecasting (Berriel et al., 2017), environmental consumption (Lee et al., 2017), airport availability forecasting (Zhu et al., 2017), etc. These studies not only proved the validity of ANNs but also demonstrated the power of ANNs in terms of time complexities and accuracy. In very recent years, the developments of various neural networks frameworks, especially recurrent neural networks, have grown even more rapidly and were popularized among different types of steamflow forecasts: flood forecasting using a encoder-decoder Long Short-Term Memory (Kao et al., 2020) and recurrent ANFIS (Zhou et al., 2019), and temperate lakes' water-level forecasting using LSTMs and ANNs (Zhu et al., 2020). These studies have further demonstrated the power of neural network frameworks and how they can be utilized as cheap alternatives or even mainstream methodologies due to their cost-efficiency and effectiveness. Due to all the merits of neural networks, in this study, we will be investigating the problem of navigable rivers' WLF using a basic, fully-connected, and self-adaptive, neural network framework, or more specifically, the development of adaptive MLPs in Nanjing navigable river. Being adaptive means the NN model will be re-built by quickly adjusting its weights from the feedback of the new data, generating adapted forecasts, resulting in more accuracy es experimented in this study. This adaptive nature of the model is indispensable and highly valuable in real life as its ability to deal with the unpredictables. For instance, if there are some factors heavily influencing today's water-level, an adaptive model can quickly gain information and adapt to this change, reconstructing the model for future accurate predictions. Furthermore, the novelty of this study will be the application of MLPs in a navigable river WLF forecast which has never been done before.

2. Methodology

2.1. Study area

Nanjing (Fig. 1) is the capital city of Jiangsu province, located along the bank of Yangtze river's downstream. It is a major city in Yangtze Delta, contributing largely to the economy, politics, culture, and information of Jiangsu Province. It is a humid subtropical region characterized by disparities between long hot summers and cold cloudy winters. Nanjing navigable river is one of the most significant Yangtze river's navigable rivers both geographically and economically. From the investigation of the Nanjing navigable river, we can gain insights of how Yangtze River's navigable rivers WLF change, providing later investigations of other Yangtze river's navigable rivers with ideas and methods. In this study, we are interested in discovering the long-term trends and accurate short-term forecasts for WLF in Nanjing navigable river. Nanjing navigable river's water-level fluctuation shows vague trend and seasonality and they oscillate tremendously between no pattern or very complex patterns. The challenge here is to learn the complex mappings between water-level fluctuation measured at different time stamps. A reasonable branching of the problem can be: 1. Univariate multi-step forecast for water-level of Nanjing navigable river, i.e. using merely Nanjing's navigable river's water-level

measurements; 2. Multivariate multi-step forecast for water-level of Nanjing navigable river, i.e. using the water-level measurements of the navigable rivers in ten cities around Nanjing city along the Yangtze river. A general workflow (Fig. 2) of our study can be structured based on this division, ranging from problem definition to final deliverable model. The former model investigates Nanjing's own temporal dependencies, whereas the latter model investigates the interactions between navigable rivers with other navigable rivers which are upstream or downstream to Nanjing navigable river.

Heatmap is a useful visualization tool invented by Cormac Kinney in 1991 to visualize the correlations between each variable. The heatmap was constructed using seaborn library (Waskom et al., 2017) in Python in Fig. 3 and the coloring represents the strength of correlations: the lighter the color, the higher the correlation. From the heatmap, we can see that Changsha's water-level is less correlated with other navigable rivers which makes sense as Changsha's precipitation is much higher than other cities. On the other hand, other cities have shown a close correlation to the city of interest, Nanjing. All ten navigable rivers' measurements were plotted in Fig. 4.

2.2. MLP: Multilayer perceptrons

MLP, colloquially referred to as the 'vanilla' neural network (Hastie et al., 2009), is a branch of feed-forward networks (Svozil et al., 1997) and is one of the first created ANNs. As the name suggests, MLP usually contains multiple layers. It mainly consists of three layers: the input layer, the hidden layer and the output layer (Rosenblatt, 1961). The number of neurons (Fig. 5) in each layer and the number of layers are often hyper-parameters and can be tuned later on to improve performance. In this study, in the output layer, the number of neurons in an output layer is equal to the number of dependent variables after the window method. Each neuron contains bias (b), input (x), and weight (w) as three main components and it also has a non-linear activation function that produces a cumulative output of the preceding neurons. Each output of a single neuron can be written as:

$$y_i = \sigma \left(\sum_i w_i x_i + b_i \right), \quad (1)$$

whereas σ is an activation function.

In this study, rectified linear unit function (ReLU),

$$\text{ReLU: } \sigma(z) = \max(0, z) \quad (2)$$

was tested and selected over sigmoid and hyperbolic tangent functions as it has the ability to deal with the Vanishing Gradient Problem, making it one of the most popular activation functions (Ramachandran et al., 2017). MLP is a gradient-descent based learning method; common selection such as hyperbolic tangent functions have gradients between $(-1, 1)$. As gradient-descent goes on, the slope and gradients of function decrease exponentially, resulting in slow learning and costly computation (Goodfellow et al., 2016).

MLP utilizes a supervised learning technique called back-propagation (Hecht-Nielsen, 1992) for training (Fig. 6). Back propagation allows the information cost function to flow back to the network in order to compute gradients, which is a fairly simple and inexpensive procedure (Goodfellow et al., 2016).

Its learning occurs between perceptrons by updating connecting weights (w). The error of each output node can be expressed as,

$$e_i = (\hat{y}_i - y_i) \quad (3)$$

where $\hat{y}_i$ is the predicted value from the neural network and root mean squared error (RMSE) will be used as the metric to measure the overall performance of the model,

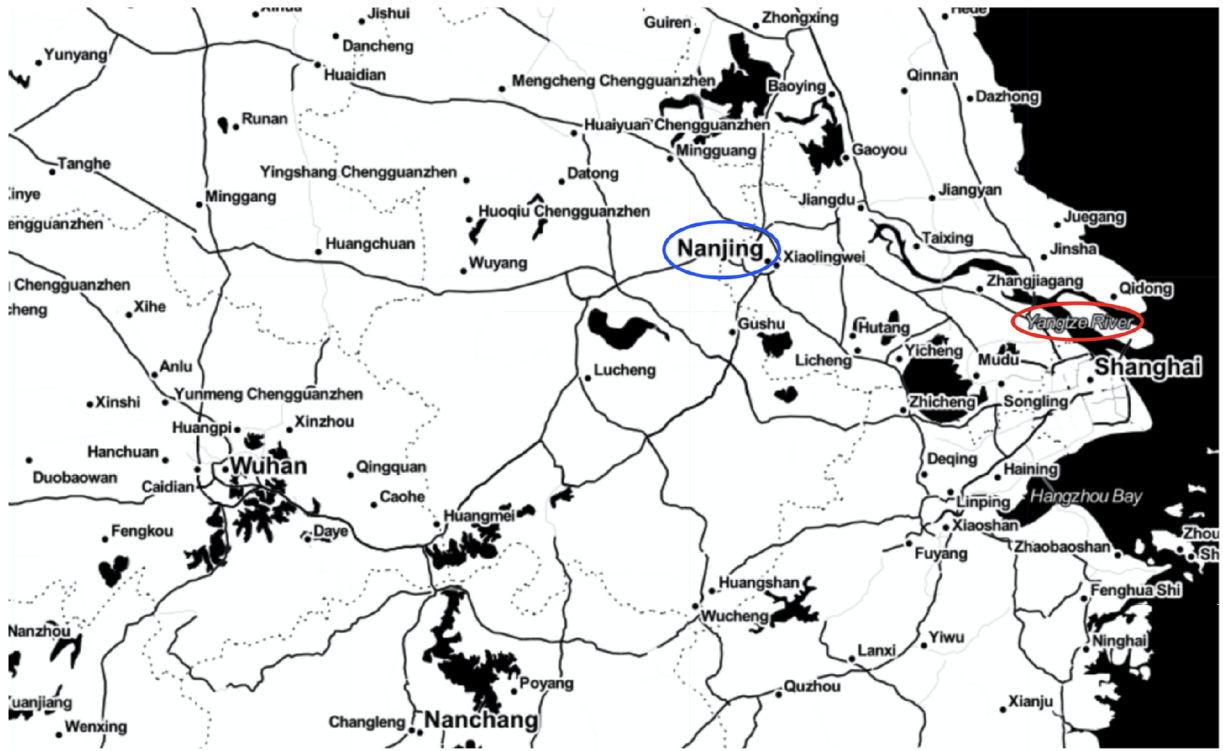


Fig. 1. Study area Nanjing at a glance.

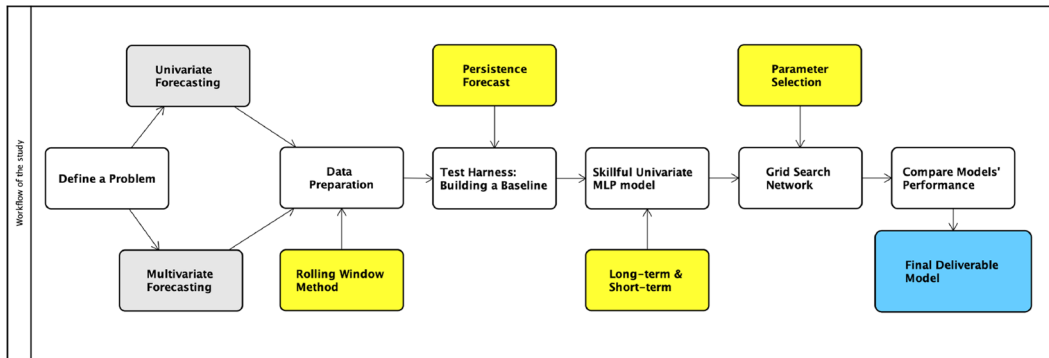


Fig. 2. A general workflow of Nanjing's navigable river time series forecast.

$$e = J(w) = \sqrt{\sum_{i=1}^n (\hat{y}_i - y_i)^2 / n} \quad (4)$$

where w is the weight between nodes that contributes to the predicted value ($\hat{y}_i$) and $J(w)$ is the loss function. To minimize $J(w)$, gradient descent is applied to update the parameters (θ):

$$w_{ij}(n+1) = w_{ij}(n) - \eta \frac{\partial}{\partial w_{ij}(n)} J(w) \quad (5)$$

where w_{ij} is the parameter between the node i and node j and η is the learning rate which is to ensure the quick converge to a good model (Haykin, 1994).

MLP has advantages and disadvantages in different settings. An issue that MLP encounters is as the number of input features increases, the algorithm will become inefficient due to its fully connected nature (Driss et al., 2017). Other neural network structures were then developed because of the issue using shared weights between neurons. In Nanjing's navigable river data, because the input size is fairly small, it's

reasonable to construct an MLP model because of the small number of total parameters. The initial dataset was cleaned by imputing miss-labeled points and divided into training (3215 days) and testing (330 days) sets by its time steps, i.e. using the oldest 3215 days or less to train the model, and the latest 330 days or less to test the model. Next, the data needs to be prepared for the model constructions.

2.3. Data preparation: Window method

The use of previous time steps to predict the next time steps is called the rolling window method (Li et al., 2014), in short, the window method or lag method, which is a common method in statistics and time series analysis. The number of previous time steps is called window width or lag size (Bontempi et al., 2012). Window method is the foundation of turning time series problems into supervised learning problems. We will apply the window method to re-frame our data for univariate and multivariate MLP models.

Suppose we have a week of data and each day (observation) has one corresponding value and we want to construct a model that has the



Fig. 3. Ten navigable rivers' heatmap. The lighter the color, the higher correlation.

ability to predict the next two days, i.e. day8 and day9. To do that, re-framing the data using previous three days, i.e. window size of 3, to predict the next two days, meaning using the values from day 1, 2, 3 as X_s and values from day 4, 5 as corresponding Y_s , using the values from day 2, 3, 4 as X_s and values from day 5, 6 as corresponding Y_s and etc (Fig. 7). The resulting matrix will have one row that contains only one Y , as our data is relatively abundant, we can delete this row as it will not impair our models' performance. For multivariate data, the window method can be analogously applied, treat each value S_i as a set of values (Tsay, 2013).

Applying this technique on the real dataset with shape [3545, 10], i.e. water-level measurements of 3545 days in 10 cities in china, is straightforward. For brevity, we will illustrate this process (Fig. 8) for a seven-day short-time forecast, i.e. $N_{output} = 7$. First of all, split the data into training set and testing set by time steps, resulting in two data-frames with shape [3213, 10] and [329, 10] (the values 3213 days and 329 days are chosen because they are divisible by 7, this is required for rolling window method). Then restructure the both the training set and the testing set into windows of size 7, resulting in two 3-D data-frames with shape [459, 7, 10] and [47, 7, 10]. Additionally, use the rolling Window Method to transform the restructured 3-D training set into a data-frame for supervised learning. The window size, i.e. the number of previous time steps to predict the next time steps, is a parameter that can be tuned later on; for now, we will call it N_{input} . If only using Nanjing's water-level data, i.e. univariate forecasting, the independent variables, X_s , will be shaped as $[\psi(N_{input}, N_{output}), N_{input}, 1]$ where ψ is a value varies with N_{input} and N_{output} (7 in this case); whereas the dependent variables, Y_s will be shaped as $[\psi(N_{input}, N_{output}), N_{output} = 7, 1]$. $\psi(N_{input}, N_{output})$ can be easily derived as,

$$\psi(N_{input}, N_{output}) = 3214 - N_{input} - N_{output} \quad (6)$$

where $3214 = \text{length of the training set} (= 3213) + 1$. For instance, if $N_{input} = 7$, $\psi(N_{input}, N_{output}) = 3214 - 7 - 7 = 3200$ from the window method. On the other hand, if using all 10 cities to predict Nanjing's water-level, i.e. multivariate forecasting, the independent variables, X_s , will be shaped as $[\psi(N_{input}, N_{output}), N_{input}, 10]$; whereas the dependent variables, Y_s will be shaped as $[\psi(N_{input}, N_{output}), 7, 10]$.

2.4. Test Harness: A baseline persistence forecast

Now the data is prepared and the following procedure is to build a baseline model to measure the validity, performance, and improvements of our MLP models. If our models achieve performance below the baseline, the model needs to be fixed or simply abandoned. A persistence forecast, or naive forecast, is a simple, fast, and repeatable method that assumes little about the nature of the forecast (Hyndman and Athanasopoulos, 2018). It can be generalized to testing possible offsets of the historical data. For instance, persisting last week's measurements, persisting last year's measurements and etc. Three models were constructed using different offsets:

- the daily persistence, uses exact measurement from one day before our forecasting period and applies this value to all forecasting period as predictions
- the weekly persistence, uses the measurements one week prior to our forecasting period as predictions to the forecasting period
- the weekly one-year ago persistence, uses one-year-ago measurements and taking one week's measurements prior to the identical week as our forecasting period

An example of 7-day forecast persistence models are demonstrated in Fig. 9.

2.5. Model construction: Building skillful MLP models

After constructing the baseline persistence models, skillful MLP models are built. A skillful model is a model that performs better than the baseline model (Brownlee, 2018). The reason to build a skillful model first, instead of tuning parameters, is to confirm the validation of the model. We don't expect the models to be the best possible but we are expecting to deliver a model that's around 80% of what can be possibly achieved. As stated in the problem definition, we will build two main kinds of models, namely univariate and multivariate, for both long-term and short-term forecasts (Fig. 10).

For the univariate case, only Nanjing water's water level data will

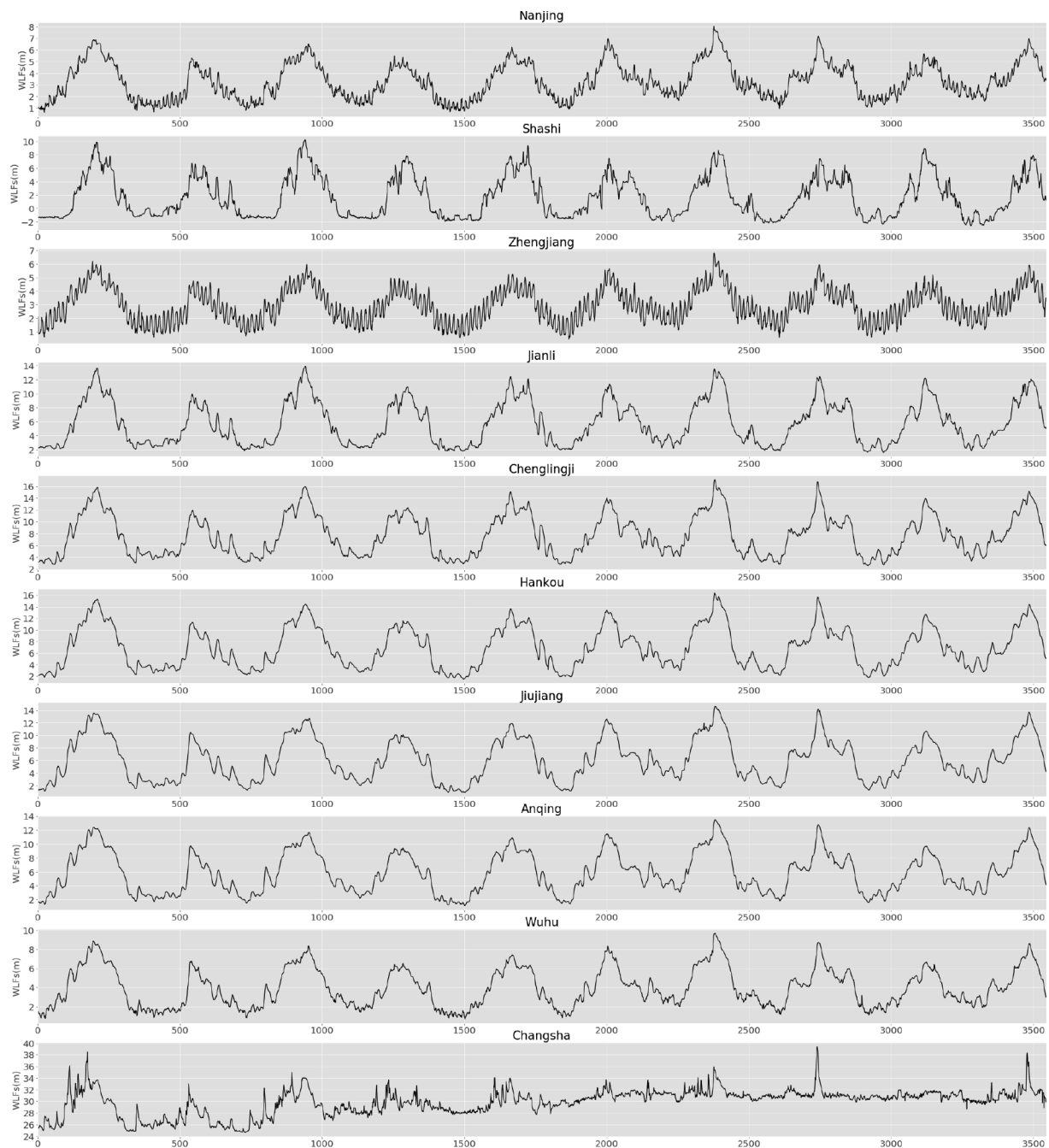


Fig. 4. Ten navigable rivers' measurements from 2010/01/01 to 2019/09/15.

be used, i.e. we did not use any other cities' measurements. The mechanics used for long-term forecasts and short-term forecasts are quite disparate. In terms of long-term forecast, we started by applying the window method on our training set and obtain the restructured dataframes, namely Train X and Train Y. Then utilized these two dataframes to fit our MLP univariate model and expect a one-day forecast when feeding the neural network with historical values in the Train X as inputs. Additionally, adding this one-day forecast to the original dataset to retrain our model, and then feeding it historical values to make another one-day forecast. The process was repeated until sufficient forecasts have been obtained, i.e. the same number of observations as the testing set. Lastly, we can compare our forecasts with the actual

measures point by point and calculate the error in metrics of RMSE. On the other hand, for short-term forecasts, the main difference is that forecasts are not added to the training set but actual measurements are added in order to update the model, resulting in an adaptive model that changes dynamically based on the actual inputs.

For the multivariate case, the ten cities' water levels will be used. These models are expected to outperform the univariate models as neural networks have the ability to distinguish complex mappings and the relationships (upstream and downstream relations) between other cities and Nanjing. The procedure is slightly adjusted from the univariate case but still adopting the idea. For long-term forecasts, repeating one-step forecasts and updating the model using Nanjing's one-

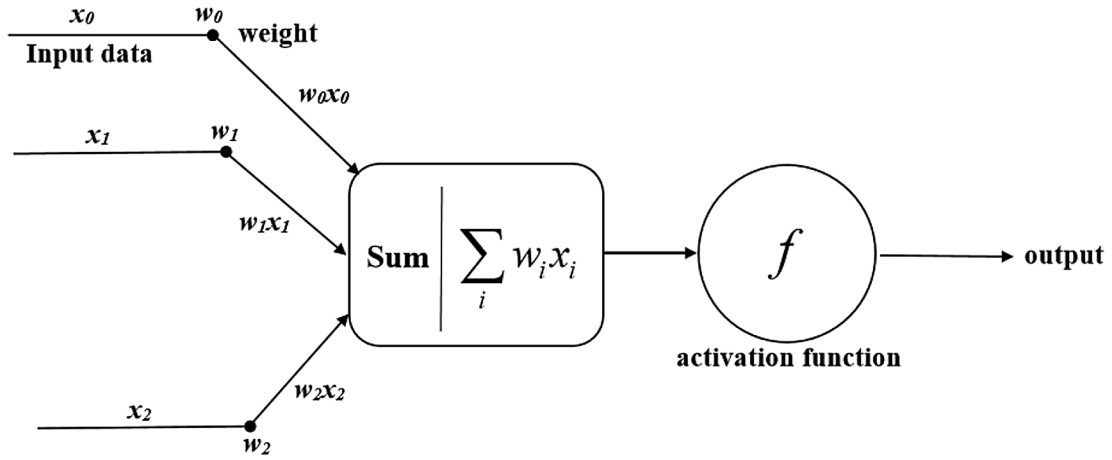


Fig. 5. Neuron model which maybe located in the hidden and output layers.

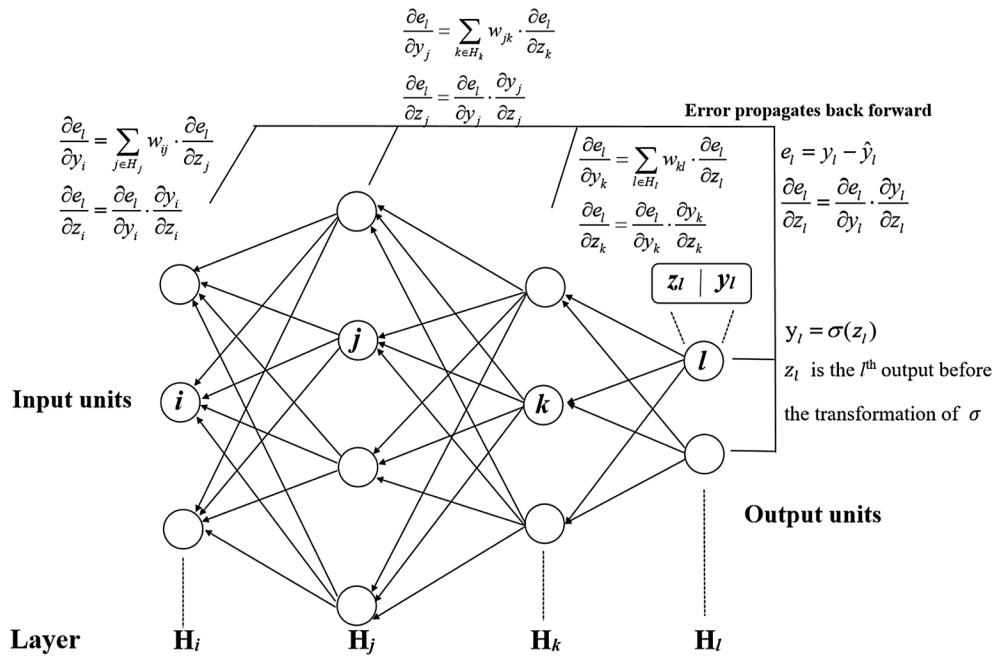


Fig. 6. A simple back propagation for gradient descent.

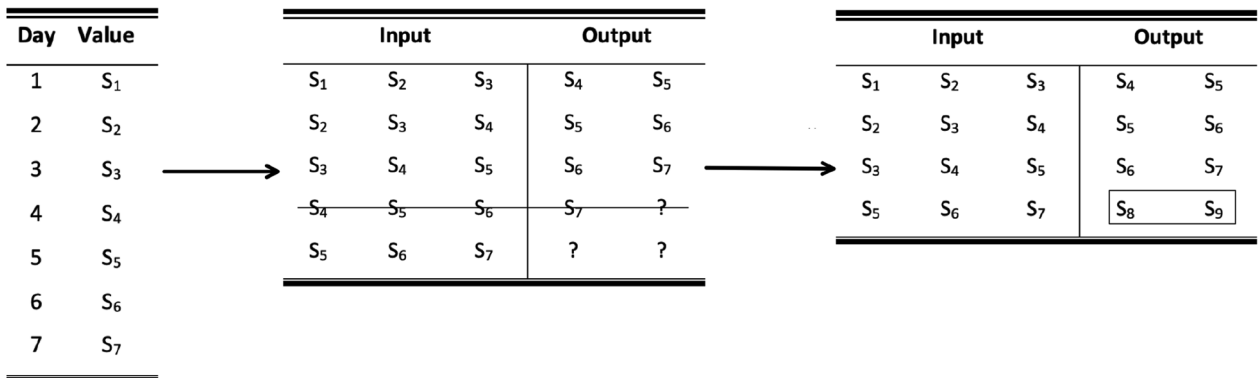


Fig. 7. Window method used on a mock univariate data.

day forecast along with actual measurements from other cities. For short-term forecasts, we are now updating the model with actual measurements from all the cities including Nanjing, also resulting in an adaptive model that changes dynamically based on the actual inputs.

One other thing to notice is that after applying the rolling window method to the training set with multiple variables, the resulting data-frame will be 3-D instead of 2-D as these variables occupy a dimension.

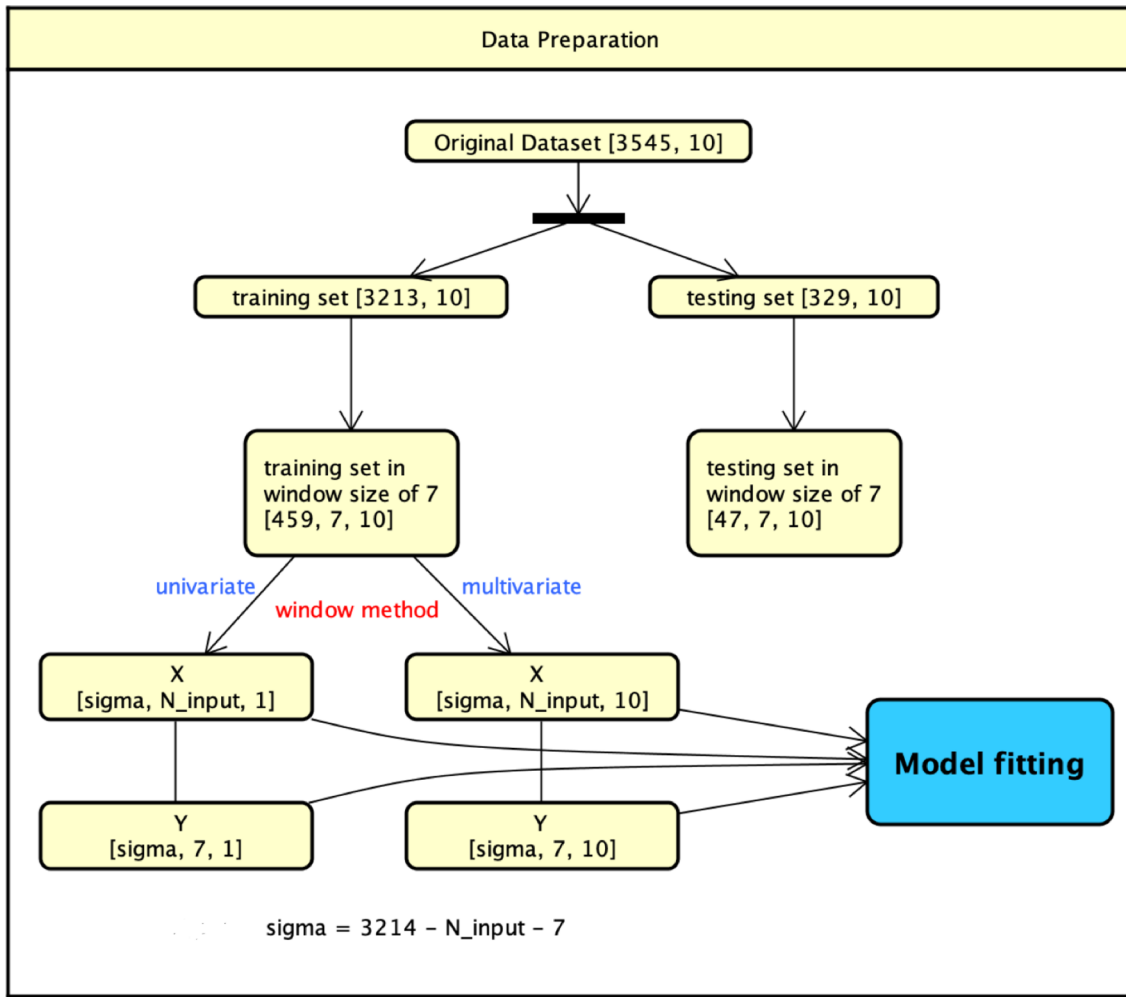


Fig. 8. Data Preparation Procedure with Window Method For a 7-day Forecast.

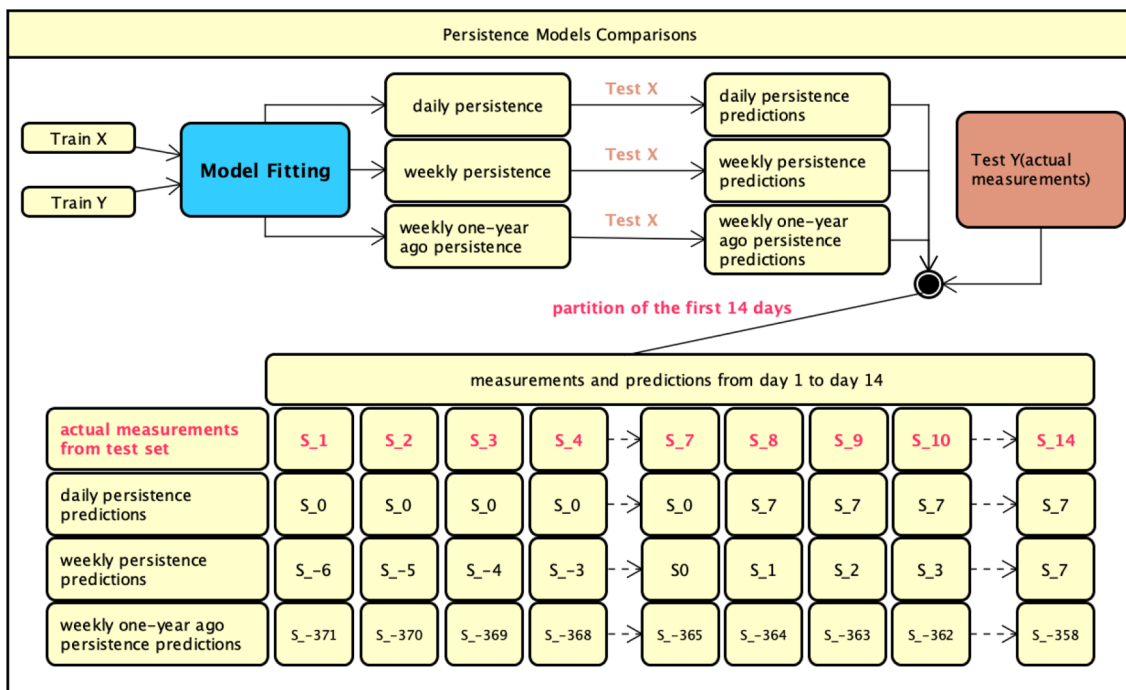


Fig. 9. Persistence Models Comparisons For a 7-day Forecast.

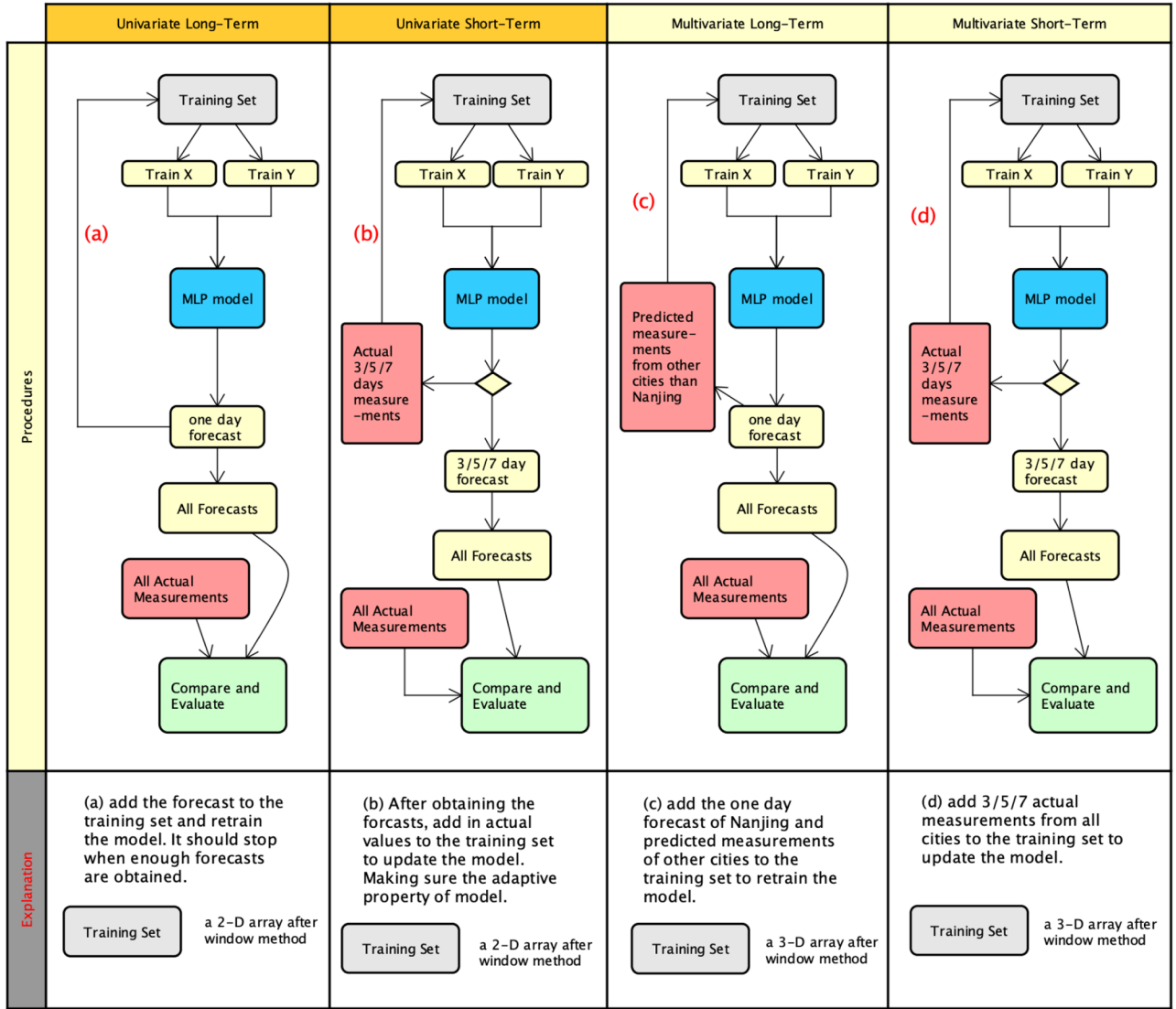


Fig. 10. Four Different MLP models for Four Scenarios.

2.6. Hyperparameter tuning: Building a grid search network

After determining the skillfulness of the models constructed earlier, we may think about ways to improve the performance or achieve the best performance of the models by tuning hyperparameters. A good set of hyperparameters is an important component of model construction. In this study, the hyperparameters selections are: N_{input} , N_{nodes} , N_{epochs} , N_{batch} , N_{hidden} .

For example, if we would like to create a univariate MLP model for a 7-day forecast (Fig. 11). We start by setting N_{input} in order to prepare the data using the window method. After that, the inputs and outputs of the neural network are now defined. We then decide how many hidden layers we are using, i.e. N_{hidden} and for each hidden layer, how many nodes (neurons) to use, i.e. N_{nodes} . Now the structure of MLP is established, we can start training the model. Picking the number of times we want to repeat this process back and forth, i.e. N_{epochs} , and decide how many samples are thrown into one mini-batch, i.e. N_{batch} .

Hyperparameter tuning is lengthy and computationally costly. With the assistance of high-performance computing clusters (HPCCs), the time complexity issue is resolved. Moreover, the set of hyperparameters

doesn't need to be tuned regularly; the same set of hyperparameters can be used without any adjustments and only the weights connecting each neuron need to be adjusted. Adjusting weights only took about 15 min on a local quad Core i5 tenth generation, which is fast and cost-efficient. Hence, we built a grid search network that takes into all combinations of the hyperparameters into account. After each run of the hyperparameters, we will re-run with the same set of hyperparameters 5 times to ensure it is not just luck due to the stochastic nature of neural networks. RMSEs of the testing set and training set from all runs were measured and were divided by the number of runs (5) as the metrics of the model performance.

3. Results

All the models with different hyperparameters were constructed and updated using Keras (Chollet et al., 2015), an API with Tensorflow (Abadi et al., 2015) back-end. The inputs and outputs of each model were obtained using the rolling window method. The hyperparameters of each model were selected from hyperparameter tuning using HPCCs in the manner of optimizing RMSE.

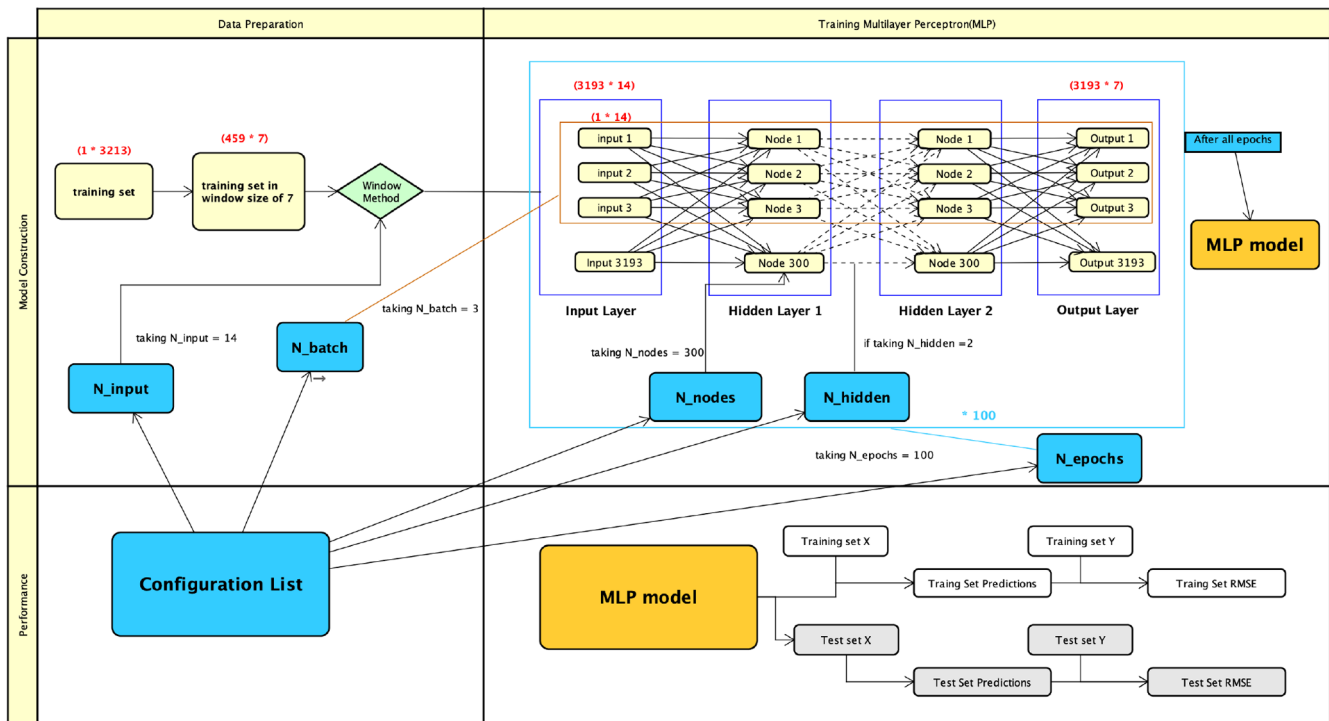


Fig. 11. An example of how each hyperparameter is used in the process of creating a Univariate MLP model for a 7-day Forecast. Repeat until all combinations of hyperparameters are tested for performance.

	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
3-day forecast	Persistence (baseline)	Daily Persisted	0.297	
		Weekly Persisted	0.903	
		Weekly one-year-ago persisted	0.934	
	Univariate MLP	200, 1000, 200, 6, 1	0.193	
		200, 1000, 200, 10, 1	0.201	
		200, 3000, 200, 6, 1	0.207	
	Multivariate MLP	150, 700, 150, 6, 1	0.161	
		100, 700, 150, 10, 1	0.176	
		100, 700, 100, 6, 1	0.177	

Fig. 12. 3-day forecast results with top 3 hyperparameters in each case.

3.1. Short-term forecasts

The short-term forecasts consist of 3, 5, and 7 days forecasting. The overall comparisons were shown in Figs. 12–14, showing the top three hyperparameters of each model after parameter tuning and the error in RMSE. For three day forecast, the baseline model, the univariate MLP and multivariate MLP achieved best RMSE of 0.297, 0.193 and 0.161, respectively. For five day forecast, the baseline model, the univariate

MLP and multivariate MLP achieved best RMSE of 0.362, 0.253 and 0.203, respectively. For a seven day forecast, the baseline model, the univariate MLP and multivariate MLP achieved best RMSE of 0.442, 0.320 and 0.229, respectively. Additionally, the best model prediction vs actual plots were also generated and zoomed in (Figs. 15–17) for better comparison.

Each scenario consisted of the baseline (persistence) model, univariate MLP, and multivariate MLP. For univariate MLP and

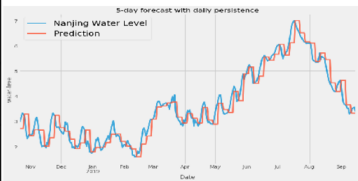
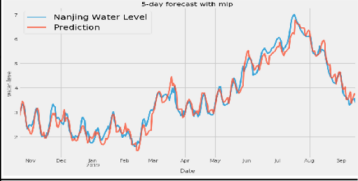
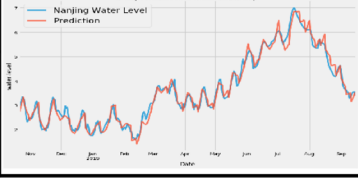
	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
5-day forecast	Persistence (baseline)	Daily Persisted	0.362	
		Weekly Persisted	1.148	
		Weekly one-year-ago persisted	1.138	
	Univariate MLP	200, 1700, 200, 6, 1	0.253	
		200, 2000, 200, 6, 1	0.263	
		150, 500, 150, 10, 1	0.275	
	Multivariate MLP	150, 1000, 200, 6, 1	0.203	
		150, 700, 130, 6, 1	0.218	
		200, 1700, 200, 5, 1	0.220	

Fig. 13. 5-day forecast results with top 3 hyperparameters in each case.

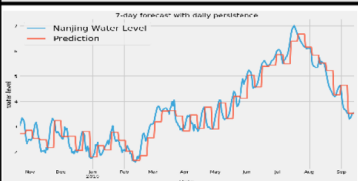
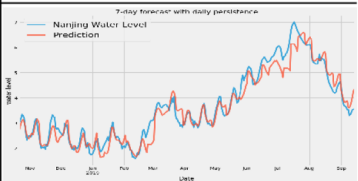
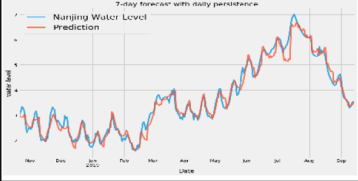
	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
7-day forecast	Persistence (baseline)	Daily Persisted	0.442	
		Weekly Persisted	1.307	
		Weekly one-year-ago persisted	0.803	
	Univariate MLP	150, 500, 200, 7, 1	0.320	
		150, 1000, 200, 10, 1	0.322	
		150, 700, 200, 7, 1	0.324	
	Multivariate MLP	150, 500, 200, 7, 1	0.229	
		200, 700, 150, 7, 1	0.245	
		100, 100, 100, 7, 1	0.250	

Fig. 14. 7-day forecast results with top 3 hyperparameters in each case.

multivariate MLP, their model hyperparameters were selected by the grid search network constructed earlier. To simplify the presentation, only the top three sets of hyperparameters of each model will be reported and the best hyperparameters' predicted values v.s actual values was plotted. We used 3213 days as the training set and predicted with a model created with the training set for the next 3/5/7 days, then added the actual 3/5/7 days' measurements to the training set and updated the model. Repeated this process and recorded each prediction cumulatively until the predictions reached the length of the testing set. Each combination of hyperparameters was repeatedly evaluated five times and each time's measurement of error (RMSE), was collected and a mean was taken as the final RMSE of the model.

In all cases (3,5 and 7 days forecasts), multivariate MLP

outperformed the other models by a lot as we expected (Fig. 18), given the non-negligible relationships between different cities. For 3-day forecasts, univariate MLP and multivariate MLP produced predictions 35.0% and 45.8% lower in terms of RMSE. For 5-day forecasts, univariate MLP and multivariate MLP produced predictions 30.1% and 43.9% lower in terms of RMSE. For 7-day forecasts, univariate MLP and multivariate MLP produced predictions 27.6% and 48.2% lower in terms of RMSE. Multilayer perceptrons or MLP models have shown great performance in short-term time series forecasts for navigable rivers' WLF.

3.2. Long-term forecast

The long-term forecast is a forecast with the length of the testing set.

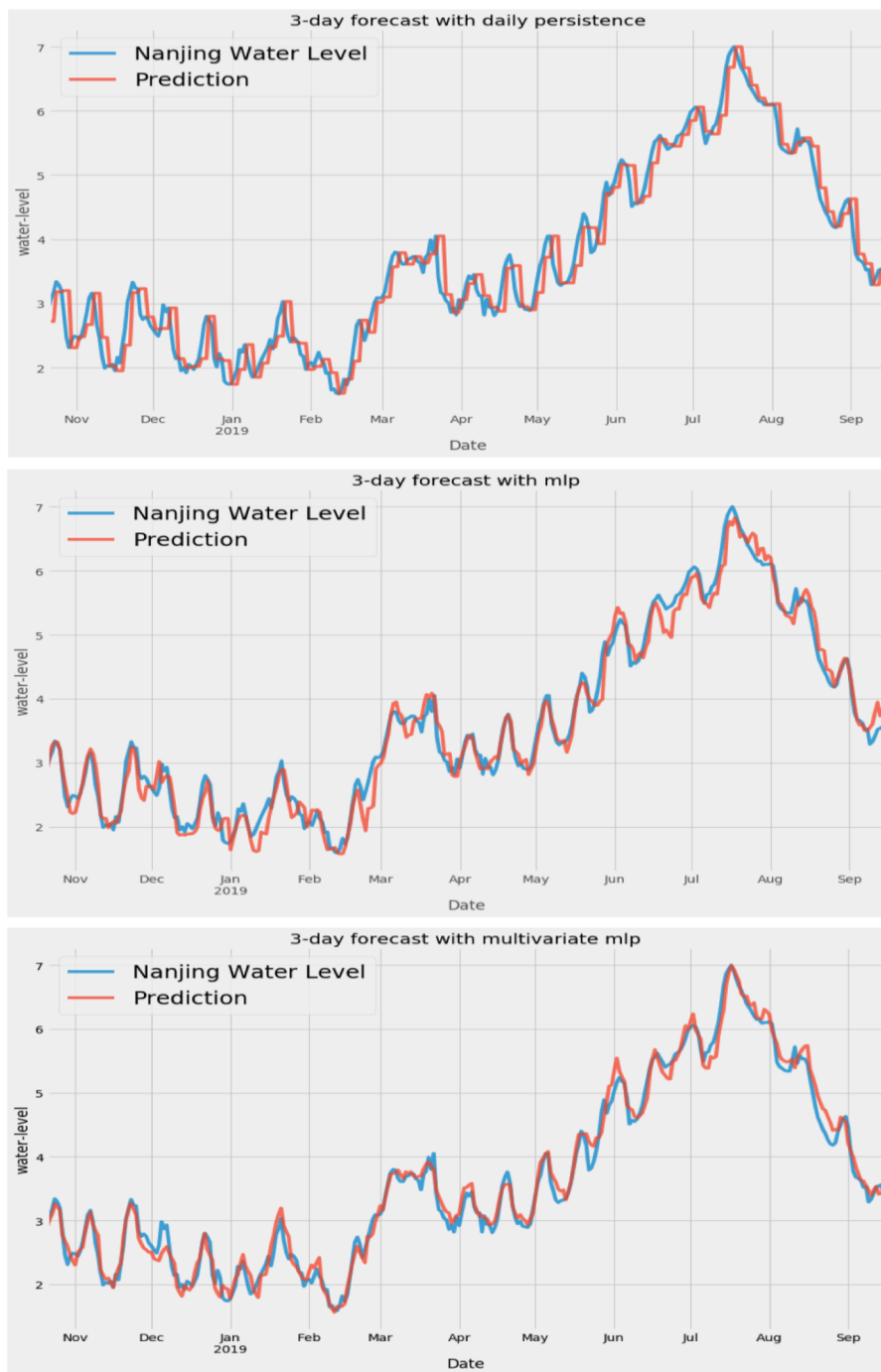


Fig. 15. Zoomed-in comparison of three best models for 3-day forecast.

In other words, we are predicting the full sequence of time series whose length is identical to the testing set's length. In this study, we predicted a full sequence of time series of length 330. The yearly persistence model, i.e. using last year's same 330 days' measurements, constructed as the baseline model to evaluate the performance of the MLP models. To summarize, for long-term forecast, the baseline model, the univariate MLP and multivariate MLP achieved best RMSE of 2.063, 1.797 and 0.659, respectively. The comparison of three models was shown in Fig. 19 with the top three hyperparameters, RMSE, and plots of results of best hyperparameters. From Fig. 19, we can see that long-term

forecasts were far from being accurate but they did exhibit the trend accurately which can be valuable in real-life settings.

The mechanics used in univariate MLP and multivariate MLP are analogous. In univariate MLP, we obtain one forecast from the constructed model and add this one forecast to the training set to construct a new model, then obtain another forecast. The process is repeated until we obtain enough forecasts whose length matches the test set's length. In multivariate MLP, we obtain ten forecasts for ten cities from the constructed model and add these ten forecasts to the training set to construct a new model, then obtain another ten forecasts. Repeat this

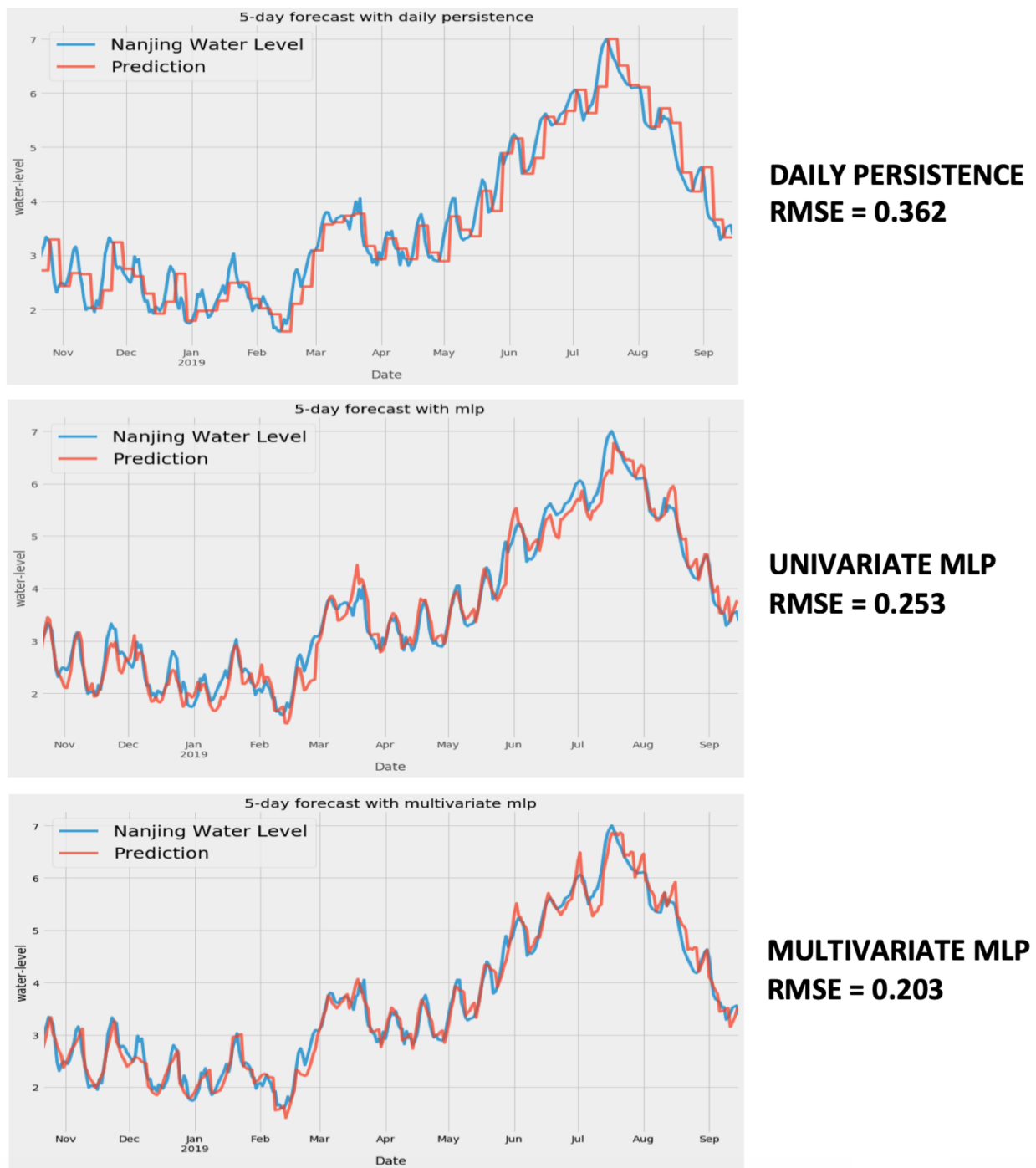


Fig. 16. Zoomed-in comparison of three best models for 5-day forecast.

process until we obtain enough forecasts whose length matches the test set's length. Upon gaining all the forecasts, we measure the performance of the model by calculating the RMSE and repeat the whole process five times due to the stochastic nature of neural networks. The final RMSE is calculated by the mean of cumulative RMSE by each trial. For brevity, we will only include the top three hyperparameters. A zoomed in comparison of the three models is shown in Fig. 20. From the plot we can see that Multivariate MLP performed so much better than univariate MLP and the baseline model for long-term forecast.

Consequently, Multivariate MLP again outperformed all the other models as we expected (Fig. 21). Univariate MLP showed a 12.9% decrease in RMSE and multivariate MLP showed a 68.1% decrease in RMSE compared to the baseline model. Unlike traditional approaches, the large-scale parameter tuning was done in less than twenty-four hours on HPCCs. Also, the univariate model and multivariate model with the selected hyperparameters only took at most 89 s and 143 s, respectively, to train and produce forecasts on a local machine with a processor of 2.0 GHz quad-core Intel Core i5 10th generation.

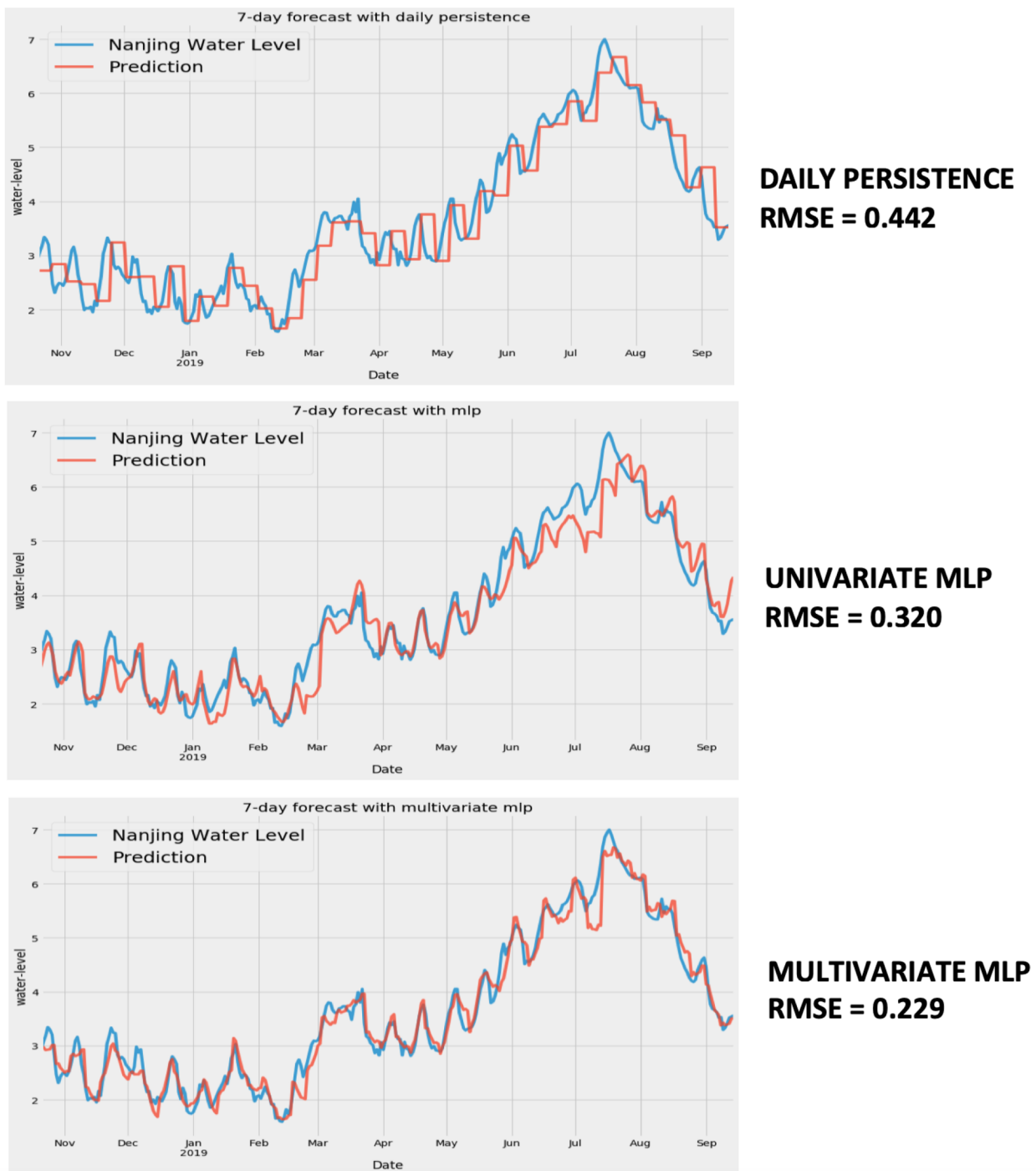


Fig. 17. Zoomed-in comparison of three best models for 7-day forecast.

4. Discussion

Overall, the utility of MLP for time series forecasting has been perfectly demonstrated. Multivariate MLP provided a valuable full sequence forecast of 330 days with only an RMSE of 0.659 and short-term forecasts with RMSEs of 0.161, 0.203, and 0.229 (3/5/7 days respectively), resulting in a final deliverable model that can be utilized in governmental navigable river regulations both for short-term and long-term forecasts. Hence, we can confidently say that it is a success of

experimenting with NNs on navigable rivers' WLF for the first time.

The results exhibited great applicability of MLP and once again proved the power of neural networks in time series problems. For short-term forecasts, univariate MLP models have shown significant improvements when compared to the baseline model, giving fairly accurate forecasts for 3/5/7 days forecasting. Additionally, multivariate MLP models provided even more improvements than univariate MLP models. For long-term forecasts, the performance of univariate MLP models are poor, perhaps due to the lack of variables, only being a little

3-day Forecast			
Model	RMSE	Relative RMSE of the baseline	Relative Lower RMSE to the baseline
Baseline	0.297	100%	0%
Univariate MLP	0.193	65.0%	35.0%
Multivariate MLP	0.161	54.2%	45.8%
5-day Forecast			
Model	RMSE	Relative RMSE of the baseline	Relative Lower RMSE to the baseline
Baseline	0.362	100%	0%
Univariate MLP	0.253	69.9%	30.1%
Multivariate MLP	0.203	56.1%	43.9%
7-day Forecast			
Model	RMSE	Relative RMSE of the baseline	Relative Lower RMSE to the baseline
Baseline	0.442	100%	0%
Univariate MLP	0.320	72.4%	27.6%
Multivariate MLP	0.229	51.8%	48.2%

Fig. 18. Performance Measurements of models in short-term forecast.

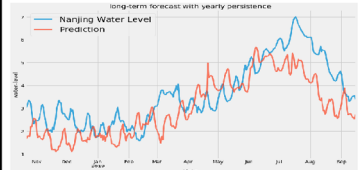
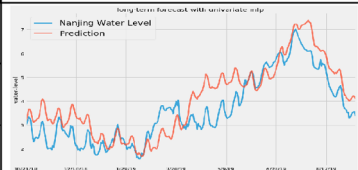
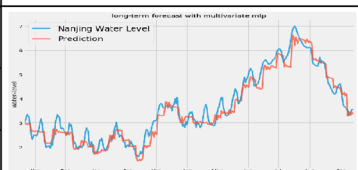
	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
Long-term Forecast	Persistence (baseline)	Yearly Persisted	2.063	
	Univariate MLP	800, 600, 200, 100, 1	1.797	
		800, 1000, 300, 20, 1	1.851	
		700, 2000, 200, 50, 1	1.852	
	Multivariate MLP	750, 2000, 200, 230, 1	0.659	
		700, 1700, 150, 300, 1	0.682	
		1000, 800, 150, 300, 1	0.689	

Fig. 19. Long-term forecast results.

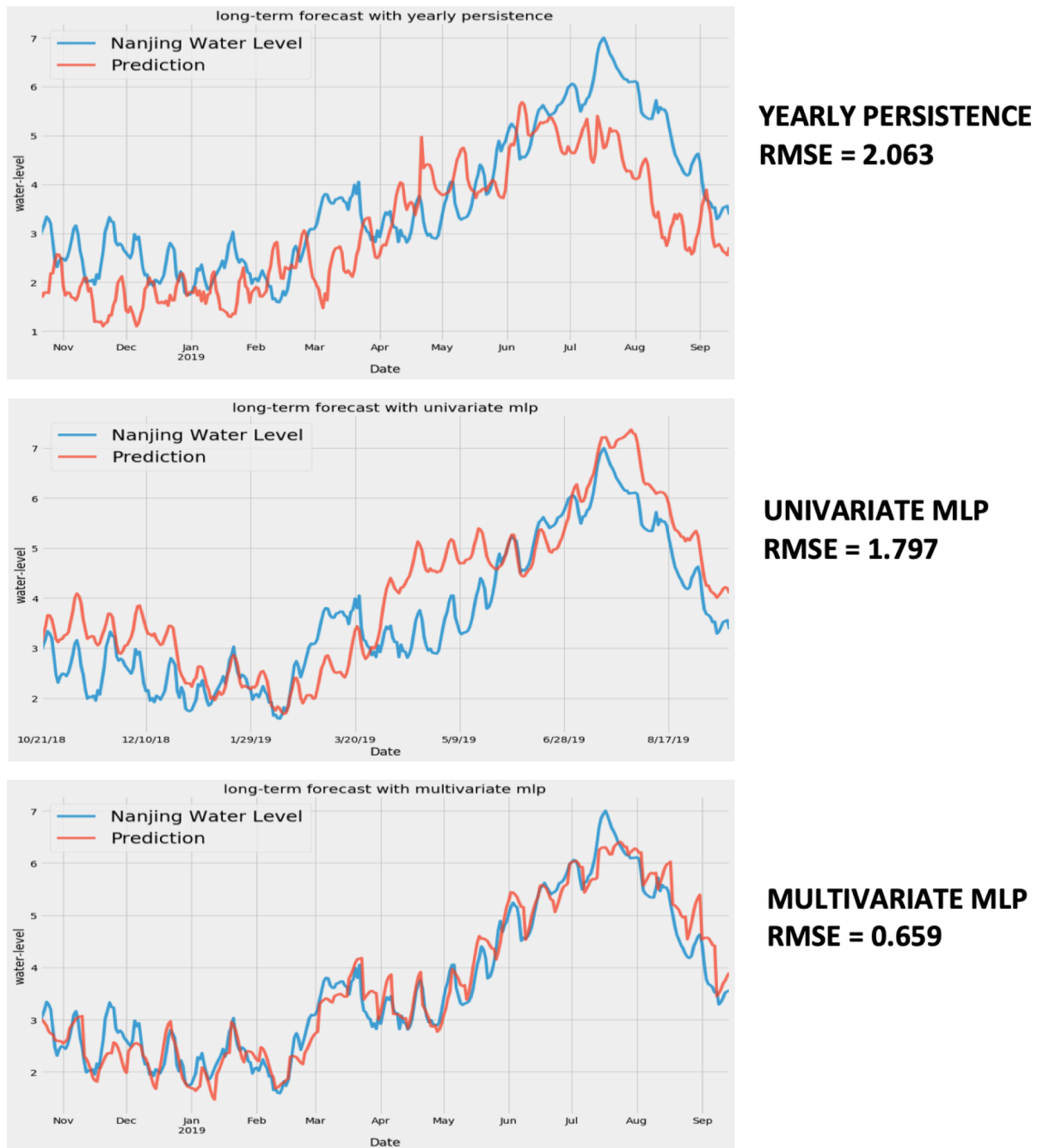


Fig. 20. A Zoomed-in comparisons of the three best models for long-term forecast.

Long-term Forecast			
Model	RMSE	Relative RMSE of the baseline	Relative Lower RMSE to the baseline
Baseline	2.063	100%	0%
Univariate MLP	1.797	87.1%	12.9%
Multivariate MLP	0.659	31.9%	68.1%

Fig. 21. Performance Measurements of models in long-term forecast.

better than the baseline model. However, the general trend and seasonality can be somehow observed. On the other hand, multivariate MLP models performed outstandingly as each neighboring, individual river has an undeniable effect on Nanjing navigable river's WLF.

Since MLP is the very basic NN and it lacks the ability to perform feature learning, which is a key attribute of CNNs and RNNs and the ability to learn temporal dependence as RNNs (Brownlee, 2018) do, it cannot generalize well in all time series problems although it performed well in this study. Additionally, due to the fully connected nature of MLP (Goodfellow et al., 2016), as the network gets larger, the number of parameters grows exponentially, resulting in overfitting. To resolve these issues, we can construct CNNs, RNNs (especially LSTM), and their mixtures, which are more complex in terms of embedding and neurons, and have better ability to prevent overfitting and perform well for time series problems.

5. Conclusions

MLP models are extremely cost-efficient, reusable, quickly implemented, and can be easily developed to a web application or software with embedded model updating mechanisms for extremely simple usage for anyone, only requiring updated measurements as inputs. Though the results are promising and validating for Nanjing navigable river, further improvements and generalization can be made using convolutional neural networks (CNNs), recurrent neural networks (RNNs), and their mixtures due to the limitations of MLP (Brownlee, 2018). MLP is, by all means, the cheap alternative or maybe a replacement for expensive traditional methodologies for navigable river WLF forecasts.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This project is sponsored by the National Natural Science Foundation of China (Grant No. 51879058) and Qing Lan Project. Special thanks to Victor Palacios for editing and polishing.

Appendix A. Description of abbreviations

The following descriptions of abbreviations are provided in alphabetical order to clarify the meaning:

- 2 – D: two-dimensional
- 3 – D: three-dimensional
- ANNs: artificial neural networks
- CNNs: convolutional neural networks
- LSTM: long short-term memories
- MLP: multilayer perceptron
- ReLu: rectified linear unit
- RMSE: root mean square error
- RNNs: recurrent neural networks
- TestX: testing set features
- TestY: testing set labels
- TrainX: training set features
- TrainY: training set labels
- WLF: water-level fluctuation

Appendix B. Notation list

The following symbols are used in this paper:

- H_n : is the n 'th layer of a neural network.
- N_{output} : is the number of next time steps to be predicted.
- N_{input} : is the number of previous time steps to predict the next time steps; this will affect the shape of Train X and Train Y (Fig. 8). it is an essential part for window method. For instance, $N_{input} = 7$ means using the previous seven days as inputs to make forecasts on the next day (s).
- N_{nodes} : is the number of nodes, or neurons, in the hidden layer, also called the size of the hidden layer. The choice of many of nodes in the hidden layer to choose is empirical. Empirically speaking, the optimal size of the hidden layer is usually between input size and output size (Heaton, 2008).
- N_{epochs} : is the number of training epochs. An epoch is a process of putting the entire training set into the neural network once, both forward and backward. The number of epochs usually determines the fitness of the model: if the number of epochs is too low, it will likely underfit; if the number of epochs is too high, it will likely overfit. So we are aiming for the middle ground for the optimum.
- N_{batch} : is the number of training samples in each batch or batch size. Due to the large size of one entire training set, the dataset is often divided into small batches. Batch size is smaller than the number of training samples.
- N_{hidden} : is the number of hidden layers in the neural network. In most cases, one hidden layer will solve most of the problems but we will try adding another layer.
- S_t : is the observed measurement on day.
- b_i : is the i 'th bias term associated with x_i .
- e_i : is the i 'th error term which is the difference between the true output and predicted output.
- e : is the total error which is the sum of each e_i .
- w_i : is the i 'th weight term associated with x_i .
- x_i : is the i 'th input.
- y_i : is the i 'th output after the transformation of σ .
- $\hat{y}_i$: is the i 'th predicted output from the model.
- z_i : is the i 'th output before the transformation of σ .
- η : is the learning rate in gradient descent.
- σ : is an activation function. It transforms the inputs in the neurons to outputs.
- ψ : is a value that varies with N_{input} and N_{output} in the window method.

Appendix C. Data availability statement

Some or all data, models, or code that support the findings of this study are available from the corresponding author upon reasonable request:

- The original dataset.
- The pre-processed dataset.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X., 2015. TensorFlow: Large-scale machine learning on heterogeneous systems, < <http://tensorflow.org/> > . Software available from tensorflow.org.
- Aksoy, H., Unal, N., Eris, E., Yuce, M., 2013. Stochastic modeling of lake van water level time series with jumps and multiple trends. *Hydrology & Earth System Sciences Discussions* 10 (2).
- Alsdorf, D.E., Melack, J.M., Dunne, T., Mertes, L.A., Hess, L.L., Smith, L.C., 2000. Interferometric radar measurements of water level changes on the amazon flood plain. *Nature* 404 (6774), 174–177.
- Altunkaynak, A., Özgür, M., Sen, Z., 2003. Triple diagram model of level fluctuations in lake van, turkey.
- Berriel, R.F., Lopes, A.T., Rodrigues, A., Varejao, F.M., Oliveira-Santos, T., 2017. Monthly energy consumption forecast: a deep learning approach. In: 2017 International Joint Conference on Neural Networks (IJCNN). IEEE, pp. 4283–4290.
- Bontempi, G., Taieb, S.B., Le Borgne, Y.-A., 2012. Machine Learning Strategies for Time

- Series Forecasting. European business intelligence summer school, Springer, pp. 62–77.
- Brownlee, J., 2018. Deep learning for time series forecasting: predict the future with MLPs, CNNs and LSTMs in Python. Machine Learning Mastery.
- Chen, X.-Y., Gu, H., 2010. Application of gray linear regression model for forecast of port throughput. *Port & Waterway Engineering* 5, 25.
- Chollet, F., et al., 2015. Keras.
- Cohen, Y., Radomski, P., 1993. Water level regulations and fisheries in rainy lake and the namakan reservoir. *Canadian Journal of Fisheries and Aquatic Sciences* 50 (9), 1934–1945.
- Coops, H., Beklioglu, M., Crisman, T.L., 2003. The role of water-level fluctuations in shallow lake ecosystems—workshop conclusions. *Hydrobiologia* 506 (1–3), 23–27.
- Driss, S.B., Soua, M., Kachouri, R., Akil, M., 2017. A comparison study between mlp and convolutional neural network models for character recognition. In: *Real-Time Image and Video Processing 2017*, vol. 10223, International Society for Optics and Photonics, 1022306.
- Fischer, P., Öhl, U., 2005. Effects of water-level fluctuations on the littoral benthic fish community in lakes: a mesocosm experiment. *Behavioral Ecology* 16 (4), 741–746.
- Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep Learning. MIT Press.
- Hastie, T., Tibshirani, R., Friedman, J., 2009. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer Science & Business Media.
- Haykin, S., 1994. Neural Networks: A Comprehensive Foundation. Prentice Hall PTR.
- Heaton, J., 2008. Introduction to Neural Networks with Java. Heaton Research Inc.
- Hecht-Nielsen, R., 1992. Theory of the backpropagation neural network. *Neural Networks for Perception*, Elsevier, pp. 65–93.
- Hyndman, R.J., Athanasopoulos, G., 2018. Forecasting: principles and practice. OTexts.
- Kao, I.-F., Zhou, Y., Chang, L.-C., Chang, F.-J., 2020. Exploring a long short-term memory based encoder-decoder framework for multi-step-ahead flood forecasting. *Journal of Hydrology* 124631.
- Kebede, S., Travi, Y., Alemayehu, T., Marc, V., 2006. Water balance of lake tana and its sensitivity to fluctuations in rainfall, Blue Nile basin, Ethiopia. *Journal of Hydrology* 316 (1–4), 233–247.
- King, C.W., Webber, M.E., 2008. Water intensity of transportation.
- Lee, D., Kang, S., Shin, J., 2017. Using deep learning techniques to forecast environmental consumption level. *Sustainability* 9 (10), 1894.
- Li, L., Noorian, F., Moss, D.J., Leong, P.H., 2014. Rolling window time series prediction using mapreduce. In: *Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014)*, IEEE, pp. 757–764.
- Li, S., He, D., 2008. Water level response to hydropower development in the upper mekong river. *AMBIO: A Journal of the Human Environment* 37 (3), 170–176.
- Piasecki, A., Jurasz, J., Skowron, R., 2017. Forecasting surface water level fluctuations of lake serwy (northeastern poland) by artificial neural networks and multiple linear regression. *Journal of Environmental Engineering and Landscape Management* 25 (4), 379–388.
- Ramachandran, P., Zoph, B., and Le, Q.V., 2017. Searching for activation functions, arXiv preprint arXiv:1710.05941.
- Rosenblatt, F., 1961. Principles of neurodynamics. Perceptrons and the theory of brain mechanisms. Report no., Cornell Aeronautical Lab Inc Buffalo NY.
- Schuermans, J., Hof, A., Dijkstra, S., Bosgra, O., Brouwer, R., 1999. Simple water level controller for irrigation and drainage canals. *Journal of Irrigation and Drainage Engineering* 125 (4), 189–195.
- Svozil, D., Kvasnicka, V., Pospichal, J., 1997. "Introduction to multi-layer feed-forward neural networks. *Chemometrics and Intelligent Laboratory Systems* 39 (1), 43–62.
- Tsay, R.S., 2013. Multivariate Time Series Analysis: With R and Financial Applications. John Wiley & Sons.
- Waskom, M., Botvinnik, O., O'Kane, D., Hobson, P., Lukauskas, S., Gemperline, D.C., Augspurger, T., Halchenko, Y., Cole, J.B., Warmenhoven, J., de Ruiter, J., Pye, C., Hoyer, S., Vanderplas, J., Villalba, S., Kunter, G., Quintero, E., Bachant, P., Martin, M., Meyer, K., Miles, A., Ram, Y., Yarkoni, T., Williams, M.L., Evans, C., Fitzgerald, Brian, C., Fonnesbeck, C., Lee, A., Qalieh, A., 2017. mwaskom/seaborn: v0.8.1 (september 2017), 542. <https://doi.org/10.5281/zenodo.883859>.
- Zhao, Z., Chen, W., Wu, X., Chen, P.C., Liu, J., 2017. Lstm network: a deep learning approach for short-term traffic forecast. *IET Intelligent Transport Systems* 11 (2), 68–75.
- Zhou, Y., Guo, S., Chang, F.-J., 2019. Explore an evolutionary recurrent anfis for modelling multi-step-ahead flood forecasts. *Journal of Hydrology* 570, 343–355.
- Zhu, L., Zhu, G., Han, L., Wang, N., et al., 2017. The application of deep learning in airport visibility forecast. *Atmospheric and Climate Sciences* 7 (03), 314.
- Zhu, S., Hrnjica, B., Ptak, M., Choiński, A., Sivakumar, B., 2020. Forecasting of water level in multiple temperate lakes using machine learning models. *Journal of Hydrology* 124819.